

Abdul Basit Sajid

Backend / Infrastructure Engineer

basit@basit.engineer | +92 324 5717425 | Remote, based in Pakistan
basit.engineer | github.com/abd-ulbasit | linkedin.com/in/abd-ulbasit

SUMMARY

Backend and infrastructure engineer, 3 years, remote for a UK team since 2023. I work on capacity, latency and the deployment path: what a system costs to run, why it is slow, and how it ships without going down. Production is Python, Node and Go on GCP/AWS with Docker; the Kubernetes and Postgres-internals depth below is self-directed.

EXPERIENCE

Software Engineer, EliteITTeam, Manchester UK (Remote)

Sept 2024 – Present

- Moved a real-time multi-model computer-vision backend from a measured ~150-concurrent ceiling to 500 under load test at 97.9% delivery, partly on four GPUs rather than one, which were only usable once the single-process bottleneck was gone. Cut per-frame inference 2,800ms → 60ms (46×) and p50 871ms → 59ms: four YOLO models onto CUDA, an $O(n^2)$ Python NMS loop replaced with `torchvision.ops.nms`, and a `broadcast=True` fan-out removed that made every response $O(N)$ in connected sockets. Found in a 7-configuration sweep in which three of my own hypotheses were falsified.
- Replaced a stop-then-start production deploy, down for a full application boot every release, with a health-gated cutover on plain Docker and no orchestrator: the replacement container starts on a transition port and traffic only moves once it passes health checks. Still the production deploy path eight months later.
- Diagnosed why detection results silently never reached mobile clients: carrier-grade NAT placed a client's two WebSocket connections on different public IPs and therefore different containers. Shipped a 684-line Go fanout proxy with SHA-256 dedup, after four alternatives were evaluated and rejected on documented grounds.
- Wrote the capacity decision document for a 60,000-concurrent-user target: corrected an internal \$206k/mo projection to \$475–570k/mo from measured per-box capacity (90 users, not the assumed 200), established that GPUs are excluded from GCP's Flexible CUD, and recommended **not** building the GPU fleet. It was not built.
- Root-caused a Stripe webhook failure and repaired the affected production data with a dry-run pass, row checksums and a written rollback procedure before executing against live records.
- Closed external penetration-test findings on two products. Shipped IP-keyed rate limiting, then caught that all traffic arrives through a single server-action IP and would bucket every user into one limiter, and re-keyed the throttler on the account instead.

Associate Software Engineer, EliteITTeam, Manchester UK (Remote)

Sept 2023 – Sept 2024

- Built and still maintain the backend for an edtech product: auth with device-seat licensing, Stripe billing with a webhook state machine, Apple and Google receipt verification, TTS and phoneme search.

OPEN-SOURCE INFRASTRUCTURE PROJECTS

pgbranch: copy-on-write Postgres branching (Go, OverlayFS, Docker)

github.com/abd-ulbasit/pgbranch

- Diagnosed why branching a 5GiB database took 61.9s and left a 5.05GiB writable layer. Postgres's pre-WAL-replay `SyncDataDirectory` opens every PGDATA file `O_RDWR`, and on OverlayFS an `O_RDWR` open of a lower-layer file forces whole-file copy-up.
- Proved the mechanism instead of arguing it: a single-variable control run with `recovery_init_sync_method=syncfs` ended at 16KiB of writable layer, not 5.05GiB. The one-flag fix takes create to 1.89s / 33MiB.
- Branching a live branch freezes the parent's write layer into a refcounted immutable layer both then share, so you can branch a database that is still being written to. A Postgres wire-protocol router reaches every branch on one port as `dbname@branch`.

upgradescope: Kubernetes upgrade-readiness scanner (Go, client-go)

github.com/abd-ulbasit/upgradescope

- Collectors degrade instead of failing: a 401/403 from a managed control plane's metrics endpoint becomes a named unavailable capability, so the report states which signal was lost rather than reporting a clean scan.
- Argued against a controller-runtime reconciler: the verdict is a function of (inventory, knowledge base, target) and two of those three change with no Kubernetes event, so a watch-driven loop requeues on unrelated pod churn yet still misses the day a component's EOL passes.

goqueue: distributed message queue (Go)

github.com/abd-ulbasit/goqueue

- Found a bucket-drain race in a hierarchical timing wheel: an element pointer held across the callback unlock is silently invalidated by a concurrent cancel, stranding every remaining timer in that bucket for up to 7.76 days. Invisible to `-race` (every access really was under the mutex) and to the existing tests; wrote deterministic regression tests that fail against the old implementation, then fixed it.

forgepoint: ML-lifecycle platform (Go, Kubernetes, gRPC, NATS JetStream)

github.com/abd-ulbasit/forgepoint

- Takes a model from training through serving and back to retraining when it drifts, as a closed loop. The engineering problem is that eleven services, each owning its own database, have to agree on what happened without a distributed transaction. That includes a cancelled workflow that still has to durably record that it was cancelled, on a context that is already dead.

WRITING, EDUCATION & CERTIFICATIONS

Bookstore Kubernetes Guide

abd-ulbasit.github.io/bookstore-kubernetes-guide

16 parts, 115 chapters: one application from a container to a production cluster.

BS Computer Science, NUST Islamabad | AWS Certified Solutions Architect (Associate), July 2025

SKILLS

Languages: Go, Python, TypeScript, SQL, Rust | **Data:** PostgreSQL, MongoDB, Redis | **Cloud:** AWS, GCP

Infrastructure: Kubernetes, Docker, Helm, Terraform, Nginx, GitHub Actions, Prometheus, gRPC, NATS JetStream

Practices: Distributed systems, concurrency, load testing and capacity planning, cost modelling, production migrations